PreExam Solution

1 Ejercicio 1: Inferencia de Tipos

Se analiza la función:

```
(define (count x l)
  (if (null? l)
      0
      (if (equal? (car l) x)
          (+ 1 (count x (cdr l)))
          (count x (cdr l)))))
```

Paso 1: Identificación de variables

- x tiene un tipo desconocido, que llamamos T.
- l es una lista de elementos del mismo tipo que x, es decir, T **list**.

Paso 2: Análisis de operaciones

- `(null? l)` requiere $l : T$ **list** y devuelve **bool**
- `(car l)` devuelve el primer elemento, de tipo T.
- `(cdr l)` devuelve la cola, de tipo T **list**.
- `equal?(car(l), x)` requiere que ambos tengan el mismo tipo, así que los dos son T, lo que confirma $x : T$.

Paso 3: Tipo de retorno

La función retorna:

- 0 en el caso base, de tipo **int**;
- (+ 1 (count ...)) en el caso recursivo, también **int**.

Ambas ramas coinciden: el tipo de retorno es **int**.

Resultado final

$$count : T \to T \textbf{ list} \to \textbf{int}$$

Patrón. Las funciones recursivas sobre listas suelen tener tipo $T \to T$ **list** $\to \alpha$. El tipo es *polimórfico* en T: funciona para listas de enteros, strings, etc.

2 Ejercicio 2: Semántica Operacional

Se desea definir la semántica de la instrucción:

$$\textbf{repeat } E \textbf{ times } C$$

Planteamiento

- La expresión E se evalúa **una sola vez** al inicio.
- El resultado n es el contador; se decrementa en 1 después de cada ejecución de C.
- Introducimos una forma auxiliar **repeat-n** k **times** C donde $k \in \mathbb{Z}$ ya es un valor entero (no una expresión).

Regla de entrada (evalúa E una sola vez)

$$\frac{\langle E, \sigma\rangle \Downarrow n \qquad \langle \textbf{repeat-n } n \textbf{ times } C, \sigma\rangle \Downarrow \sigma'}{\langle \textbf{repeat } E \textbf{ times } C, \sigma\rangle \Downarrow \sigma'} \quad [\textsc{Repeat-Eval}]$$

Caso base ($k \leq 0$)

$$\frac{k \leq 0}{\langle \textbf{repeat-n } k \textbf{ times } C, \sigma\rangle \Downarrow \sigma} \quad [\textsc{Repeat-Fin}]$$

El caso base usa $k \leq 0$ y no $k = 0$. Si E da un número negativo y solo existe el caso $k = 0$, ninguna regla aplica y la configuración queda bloqueada. Con $k \leq 0$, `repeat -3 times C` no hace nada, como un `for` cuyo rango está vacío.

Caso recursivo ($k > 0$)

$$\frac{k > 0 \qquad \langle C, \sigma\rangle \Downarrow \sigma' \qquad \langle \textbf{repeat-n } (k{-}1) \textbf{ times } C, \sigma'\rangle \Downarrow \sigma''}{\langle \textbf{repeat-n } k \textbf{ times } C, \sigma\rangle \Downarrow \sigma''} \quad [\textsc{Repeat-Step}]$$

Corrección. En la regla Repeat-Step, *no* se vuelve a evaluar E. Solo se trabaja con el contador entero k y se resta 1. Re-evaluar E en cada iteración violaría la especificación del enunciado.

Intuición: equivalente a un `for`-loop donde el límite superior se fija antes de entrar al ciclo.

3 Ejercicio 3: Traducción Infix a Postfix

Función semántica

Definimos $\mathcal{P}[\![\cdot]\!] : \textbf{Expr} \to \textbf{String}$ que traduce una expresión infija a postfija.

Reglas denotacionales

$$\mathcal{P}[\![n]\!] = \texttt{"n"}$$
$$\mathcal{P}[\![E_1 + E_2]\!] = \mathcal{P}[\![E_1]\!] \frown \mathcal{P}[\![E_2]\!] \frown \texttt{"+"}$$
$$\mathcal{P}[\![E_1 - E_2]\!] = \mathcal{P}[\![E_1]\!] \frown \mathcal{P}[\![E_2]\!] \frown \texttt{"-"}$$
$$\mathcal{P}[\![E_1 * E_2]\!] = \mathcal{P}[\![E_1]\!] \frown \mathcal{P}[\![E_2]\!] \frown \texttt{"*"}$$
$$\mathcal{P}[\![E_1/E_2]\!] = \mathcal{P}[\![E_1]\!] \frown \mathcal{P}[\![E_2]\!] \frown \texttt{"/"}$$

donde $\frown$ denota concatenación de strings (con espacio separador).

Ejemplo: $3 + 4 * 5$

Con precedencia estándar ($*$ liga más que $+$), el árbol es $3 + (4 * 5)$:

$$\begin{aligned}
\mathcal{P}[\![3 + (4 * 5)]\!] &= \mathcal{P}[\![3]\!] \frown \mathcal{P}[\![4 * 5]\!] \frown \texttt{"+"} \\
&= \texttt{"3"} \frown (\mathcal{P}[\![4]\!] \frown \mathcal{P}[\![5]\!] \frown \texttt{"*"}) \frown \texttt{"+"} \\
&= \texttt{"3"} \frown \texttt{"4"} \frown \texttt{"5"} \frown \texttt{"*"} \frown \texttt{"+"} \\
&= \texttt{"3 4 5 * +"}
\end{aligned}$$

Regla. Izquierdo $\frown$ derecho $\frown$ operador. La precedencia la maneja el *parser* al construir el AST; la función $\mathcal{P}$ solo recorre el árbol resultante.

4 Ejercicio 4: Alcance Estático vs. Dinámico

Los dos modelos

Modelo	Pregunta a hacer	Resultado
Estático (léxico)	¿Dónde está *definido* `proc1` en el fuente?	$x = 2$
Dinámico	¿Qué hay en la *pila de llamadas* ahora mismo?	$x = 3$

Alcance estático

`proc1` está definido en el **nivel global** del programa. El único x visible en su contexto de definición es el x global, cuyo valor es 2 (asignado por `main` antes de llamar a `proc2`).

Por eso el `print` imprime `x=2`.

Alcance dinámico

En tiempo de ejecución, la pila al momento del `print` es:

$$\underbrace{\texttt{main}}_{x=2} \longrightarrow \underbrace{\texttt{proc2}}_{x=3\ \text{(local)}} \longrightarrow \texttt{proc1}$$

El enlace más reciente de x en la pila es el de `proc2`, con valor 3.

Por eso el `print` imprime `x=3`.

Regla mental. Estático = dónde se *define* (contexto léxico)
Dinámico = desde dónde se *llama* (pila en tiempo de ejecución)

5 Ejercicio 5: Paso de Parámetros

Tipos de parámetros

- **ref** (por referencia): alias directo al argumento. Leer/escribir el parámetro lee/escribe la variable original.
- **name** (por nombre / Jensen's device): la expresión argumento se *re-evalúa* cada vez que se usa el parámetro. Si las variables internas cambian, cada uso puede acceder a una celda diferente.
- **VR** (value-result): copia *in* al entrar ($z_{\text{local}} \leftarrow$ valor actual del argumento) y copia *out* al salir ($i \leftarrow z_{\text{local}}$).

Estado inicial

$$a = \{0,\ 10,\ 20,\ 40\}, \qquad i = 0$$

Llamada: `test(i, a[i], i)`

- $x \xleftarrow{\text{ref}} i$: alias de i
- $y \xleftarrow{\text{name}}$ `a[i]`: expresión diferida
- $z_{\text{local}} \xleftarrow{\text{VR}} 0$ (copia-in de $i = 0$)

Ejecución línea a línea

Línea	Sentencia	Efecto	Estado tras ejecutar
8	`x := x+1`	x es alias de i; $i \leftarrow 0{+}1 = 1$	$i = 1,\ z_{\text{local}} = 0$
9	`y := y+1`	$y =$ `a[i]` con $i = 1$, así que $y = a[1] = 10$	$a[1] \leftarrow 11$
10	`z := z+2`	copia local $z_{\text{local}} \leftarrow 0{+}2 = 2$	$z_{\text{local}} = 2,\ i = 1$
11	`y := y+1`	$y =$ `a[i]` con $i = 1$ todavía, así que $y = a[1] = 11$	$a[1] \leftarrow 12$
Retorno (VR copia-out)		$z_{\text{local}} = 2$, y se copia: $i \leftarrow 2$	$i = 2$

Resultado final

$$i = 2, \qquad a = \{0,\ 12,\ 20,\ 40\}$$

Tres interacciones críticas.

1. **ref cambia el índice de name.** La línea 8 incrementa i de 0 a 1 vía `ref`. Las líneas 9 y 11 usan $y = a[i]$ con $i = 1$, accediendo a $a[1]$ (no $a[0]$).

2. **name modifica la misma celda dos veces.** Tanto la línea 9 como la 11 operan sobre $a[1]$, incrementándolo de 10 a 11 y luego a 12.

3. **VR sobreescribe el cambio de ref al retornar.** `ref` dejó $i = 1$, pero la copia-out de VR escribe $z_{\text{local}} = 2$ sobre i, resultando en $i = 2$.